

Speaking Notes — Conversational Travel Assistant

Structure: explain all 4 topics first, then run the full demo in Colab

Everything under "SAY THIS" is written to be spoken almost as-is — read it in your own voice, don't recite it robotically. Bracketed lines are stage directions for you, not for the class. Tied to `travel_assistant.ipynb`.

Opening (≈1-2 min)

SAY THIS:

"Hi everyone, welcome. Today we're going to build a small travel assistant together — one that can check the weather in a city, tell you what's worth seeing there, and search flights between cities. It's going to run on completely fake data, so nothing here depends on a live API going down mid-class.

Before we touch any code, I want to walk you through four ideas that make this whole thing work. Once those four ideas are clear, the code will basically explain itself — you'll be able to look at almost any line and know which of the four ideas it belongs to. So let's start there, and then we'll go build it live in Colab."

TOPIC 1 — How do we connect to the model? (≈3-4 min)

SAY THIS:

"First question: when we say our code 'talks to the model,' what does that actually mean? The model itself — Grok, in our case — isn't running on your laptop or inside Colab. It's running on xAI's servers, somewhere else entirely. So 'connecting to the model' really means three things:

One — we need to prove who we are. That's what an API key is. Think of it like a phone number and a password combined — it identifies your account and lets the provider bill you for what you use.

Two — we need an object in our code that knows how to make that phone call. In LangChain, that's a *chat model object*. We're going to create one called `llm`, using a class called `ChatXAI`. All that object does is package up your message, send it over the network to Grok, and hand you back the reply.

Three — and this is the part I want you to really hold onto — this object is interchangeable. If tomorrow you wanted to use a different model provider instead of Grok, you would change *one line* — the line that creates this object — and nothing else in your code would need to change. The rest of your application doesn't know or care which company is answering on the other end of that phone call.

One practical note before we get to the demo: unlike some providers, Grok doesn't have a permanent free tier. When you sign up at console.x.ai, you get a batch of free trial credits — historically around \$25 — which is more than enough to run this entire class many times over. So 'free' here means 'free trial credits,' not 'free forever.' I'm flagging that so nobody's surprised later."

[Transition]: "That's connecting to the model. Next — what happens when the model forgets what you told it thirty seconds ago?"

TOPIC 2 — Multi-turn conversation with memory (≈3-4 min)

SAY THIS:

"Here's something important about how these models work: every single request you send is stateless. That means the model has no memory of anything you said before, unless you explicitly send it again.

Picture this: you ask 'what's the weather in Jaipur?' and get an answer. Then you ask 'what can I see there?' If all we send the model is that second sentence on its own, it has no idea what 'there' refers to. It wasn't in the room for your first question — every request starts from a blank slate.

So how do chatbots seem to remember a conversation? The trick is simple, and it's worth saying plainly because it demystifies a lot of what looks like magic: we keep a running transcript of everything said so far — every user message, every model reply — and we resend that entire transcript with every single new question. The model isn't remembering anything. We are reminding it, in full, every single time.

We're going to represent that transcript as a plain list in Python — a list that starts empty and grows by a few messages after every turn. When you see `conversation_history` in the code shortly, that list is the memory. There's no hidden magic object doing this for us — we're building it by hand so you can literally watch it grow."

[Transition]: "So we know how to talk to the model, and how to help it remember. Now — how do we structure the actual flow of 'user says something, response comes back'? That's a chain."

TOPIC 3 — Chains (≈3-4 min)

SAY THIS:

"You've already built simple chains before, so I won't re-teach the whole idea — but let's be precise about what a chain buys us here.

A chain is a fixed pipeline: some input goes in one end, it passes through a series of steps in order, and a result comes out the other end. In our case the pipeline is short: take the user's message and the conversation history, format them into a proper prompt using a prompt template, and pass that prompt into the model. In LangChain we write that as one line — a

prompt piped into a model — and that pipe symbol is doing exactly what it looks like: the output of the first step becomes the input to the second.

Why bother with this instead of just calling the model directly? Two reasons. First, it keeps the *shape* of our prompt consistent — the system instructions, the history, and the new message always get assembled the same way, every single time, without us retyping that structure by hand. Second, it gives us one clean object — the chain — that we can call the exact same way regardless of what's happening underneath. That consistency is what let us bolt memory and tools onto this without rewriting everything.

One detail to watch for when we get to the notebook: inside the prompt, there's a placeholder — an empty slot — reserved specifically for the conversation history we talked about in Topic 2. That's the seam where memory and chains connect to each other."

[Transition]: "Last idea, and it's the one that makes this feel like an actual assistant instead of just a chatbot: how does the model check real weather data, or search real flights, when all it can do is generate text?"

TOPIC 4 — Calling tools (~4-5 min)

SAY THIS:

"Here's the core problem: a language model can only produce words. It cannot reach into a database, it cannot run a calculation, it cannot look anything up. So how does an assistant like this one 'check the weather'?"

It doesn't — we do, on its behalf. Here's the analogy I want you to keep in your head for the rest of this class: think of the model as a waiter, not a chef. A customer — our user — asks for something. The waiter doesn't walk into the kitchen and cook it themselves. They take a precise order back to the kitchen — that's our code — wait for the kitchen to prepare it, and bring the finished dish back out to the table, described in plain language.

In our app, the 'kitchen' has three dishes it can make: check the weather for a city, list attractions in a city, and search flights between two cities. Each of those is just an ordinary Python function that reads from one of our fake datasets. Nothing fancy — normal code you could write and test on its own.

The only new part is how the model finds out those functions exist. We wrap each function with something called a *tool* decorator, which reads the function's name, its description, and what inputs it expects, and turns that into a little menu card. Then we hand that menu to the model — that step is called *binding* — so that on every request, the model knows: 'here are three things you're allowed to ask for, and here's exactly what each one needs.'

Here's the key mental model, and I want to say it very precisely: the model *decides* whether a tool is needed and *requests* it by name with specific arguments. Our code is the one that actually *runs* the function and sends the real result back. The model never executes

anything itself — it only ever asks, in a structured way, and then turns whatever comes back into a natural sentence for the user."

[Transition]: "So — connect to the model, give it memory, wrap it in a chain, and let it call tools when it needs real data. That's the whole architecture. Let's go build every piece of that, live, in Colab."

THE DEMO (≈20–25 min)

Narrate every cell as you write or run it. Each heading below names which of the four topics that cell demonstrates, so you can call back to the theory explicitly as you go.

Demo §1 — Setup [Topic 1: connecting to the model]

SAY THIS: "Let's start by installing the LangChain packages, plus `langchain-xai`, which is the official connector for Grok. Then I'll paste in my API key — never hardcode this in a real notebook, always type it in live or pull it from Colab's secrets manager."

[Install, then run the getpass cell.]

Demo §2 — Four quick examples, before we build the real thing

SAY THIS: "Before we build the actual assistant, I want to show each of the four ideas on its own, with the smallest possible example. None of this code ends up in the final app — it's scratch work so each idea is crystal clear before we combine all four."

§2.1 — Connecting to the model [Topic 1]

SAY THIS: "Here's the whole connection, stripped down to nothing else. Create the `ChatXAI` object, call `.invoke()` with a single string, print `.content`." *[Run it.]* "That's it — that's the phone call from Topic 1, with nothing else attached yet."

§2.2 — Multi-turn memory [Topic 2]

SAY THIS: "Now memory, in isolation. I'm going to ask the exact same question — 'What can I see there?' — two different ways."

[Run the no-history cell.] "First, with nothing before it. Watch how vague or generic this answer is — the model has no idea what 'there' means."

[Run the with-history cell.] "Now, same question, but I've put a short made-up exchange about Jaipur in front of it first. Same final question, completely different quality of answer. That's the entire mechanism — whether we included that list of earlier messages or not."

§2.3 — A chain [Topic 3]

SAY THIS: "Now a chain, with nothing travel-related in it at all, so you can see the *shape* clearly. This one rewrites a sentence in a given tone." *[Run it.]* "Three pieces piped together: the prompt template, which fills in `tone` and `sentence`; the model; and a parser on the end that just unwraps the plain text so we get a string back instead of a full message object. Let's

change the tone to something else and rerun — "*[optionally re-run with a different tone]*" — same chain, different input, different output. That's the whole idea of a chain: a fixed pipeline you can reuse."

§2.4 — Calling a tool [Topic 4]

SAY THIS: "Last one — a tool that has nothing to do with travel: adding two numbers. I picked arithmetic on purpose, because models are famously unreliable at large-number math, which makes it obvious when real code is doing the work instead of the model guessing."

[Run the bind + invoke cell.] "Look at what came back — the model didn't answer in words at all. `tool_response.tool_calls` shows it made a structured request instead: 'call add_numbers with these two numbers.'"

[Run the execute cell.] "And here's us actually running the real function with those arguments. This line — `add_numbers.invoke(...)` — is our code, not the model's. Request, then execute. That's tool calling, completely stripped down."

SAY THIS (closing §2): "Four ideas, four tiny examples. Now let's combine all of them into the real assistant."

Demo §3 — The fake data

SAY THIS: "Let's look at the assistant's entire world — three plain Python structures: a weather dictionary, an attractions dictionary, and a list of flights. No live API anywhere. Our tools are about to read straight from these."

[Paste and run the three data structures.]

Demo §4 — Defining tools [Topic 4: calling tools]

SAY THIS: "Now let's build the 'kitchen' from our waiter analogy. I'll write `get_weather` first." *[Write it live.]* "Notice the docstring — that's not just a comment for humans, it's the description the model will read to decide when this tool is useful. The `@tool` line above the function is the only non-ordinary part here — everything below it is plain Python you could run and test with no model involved at all."

[Repeat for `get_attractions` and `search_flights`, faster each time.]

"Last, I'll put all three into a list called `tools`, and a lookup dictionary by name — we'll need that lookup in a few minutes, to go from 'the model asked for a function called get_weather' to 'here's the actual function to run.'"

Demo §5 — Connecting to the model [Topic 1]

SAY THIS: "Here's the connection itself. `ChatXAI` is the object we talked about — it's our phone line to Grok. I'm using `grok-4-fast` because it's inexpensive per token, which matters if we're running on trial credits. And here's the line that ties Topic 1 to Topic 4:

`bind_tools`). This does *not* let the model execute anything — it only hands the model that menu card of what it's allowed to ask for."

Demo §6 — Building the chain [Topic 3]

SAY THIS: "Now the chain. We build a prompt template with a system message, and — here's the piece from Topic 2 — a placeholder reserved for conversation history. Then: prompt, piped into our tool-bound model. That's the whole chain. One line. Everything before this was building the ingredients; this line is where they actually get connected."

Demo §7 — The history list [Topic 2]

SAY THIS: "And here's memory, exactly as plain as I described it — an empty list.

`conversation_history = []`). Nothing clever yet. Watch it grow."

Demo §8 — The `chat()` function [Topics 2, 3, and 4 together]

Slow down here — this is the payoff of all four topics at once.

SAY THIS: "This function is the full loop for one turn of conversation, and it uses everything we just talked about. Let's read it together.

First, we call our chain — Topic 3 — with the new input *and* the running history — Topic 2.

Then we check: did the model ask for a tool? If it did — this is Topic 4 — we go through each request, find the matching function in our lookup dictionary, and actually run it with real arguments. That's the waiter walking to the kitchen. We take the result and hand it back to the model as a special kind of message, then call the chain *again* so the model can turn that raw result into an actual sentence — the waiter bringing the dish to the table and describing it.

If the model didn't ask for a tool at all, we skip straight to using its plain text reply.

Either way — and this is the line that makes Topic 2 real — we add everything that just happened onto `conversation_history`. That's the only reason the next question will have any context at all."

Demo §9 — Running a live conversation

SAY THIS: "Let's actually talk to it." [Run each `chat(...)` call one at a time, reading the output aloud.]

"First question — weather in Jaipur." [Run, read result.]

"Now — 'what can I see there?' Watch closely: I never said Jaipur in this sentence." [Run, read result. Pause.] "How did it know 'there' meant Jaipur? Nothing in this message says so — that context came entirely from `conversation_history`, exactly what we discussed in Topic 2."

"Now let's ask about flights from Delhi to 'there' — again, no city named directly." [Run.]

"This one's combining facts from two turns back — the tool call here needed both the

destination we established earlier *and* a new detail, Delhi, from this message."

"One more — weather in Goa, in case the trip gets extended." *[Run.]*

Demo §10 — Inspecting history

SAY THIS: "Let's print the raw history list so you can see the trick laid completely bare." *[Run, scroll through it.]* "This is it. No hidden state anywhere else in this program — just this list, growing turn by turn, and getting resent in full every single time we call the chain."

Demo §11 — Wrap-up

SAY THIS: "To close: everything we built today maps onto the four ideas from the start of class. `ChatXAI` connected us to the model. `conversation_history` gave it memory. The prompt-into-model pipe was our chain. And `bind_tools` plus our three functions let it act on real — well, fake, but real-*shaped* — data.

If you wanted to take this further: LangChain has a built-in helper called `RunnableWithMessageHistory` that manages that history list for you automatically — now that you've seen what it's doing by hand, that shortcut won't feel like a black box. And adding a fourth capability, like hotel search, is just one more function with a `@tool` decorator added to the `tools` list — nothing else in this chain needs to change."

Timing Cheat Sheet

Segment	Target time
Opening	1-2 min
Topic 1 — Connecting to the model	3-4 min
Topic 2 — Multi-turn memory	3-4 min
Topic 3 — Chains	3-4 min
Topic 4 — Calling tools	4-5 min
Theory subtotal	≈14-19 min
Demo §1 (setup)	≈2 min
Demo §2 (four quick examples, §2.1-§2.4)	≈6-7 min
Demo §3-§5 (data, tools, connect for the real app)	≈6 min
Demo §6-§8 (chain, history, chat() walkthrough)	≈8 min
Demo §9-§11 (live run, inspect, wrap-up)	≈6-8 min
Demo subtotal	≈28-31 min
Total	≈42-50 min — trim §2 or the live-run turns if you need to hit 40